

ENGINEERING STATUS REPORT

MetaGlasses Backend

Current architecture, delivered capabilities, security posture, operational constraints, and prioritized path to production.

Report date	September 4, 2026
Version	0.1.0
Baseline commit	a88f3ef560977977e4a97d59cf16946686f1c0c1
Repository	github.com/Adipat2003/metaglasses-backend
Scope	FastAPI backend and delivery pipeline

Prepared from the verified repository state, local runtime, automated tests, GitHub configuration, official Supabase documentation, and official OpenAI documentation.

Executive summary

The MetaGlasses backend is a functional v0.1 FastAPI service for a phone-to-lens text-assistance loop. It accepts a phone-maintained conversation transcript, asks OpenAI for one concise next step, and exposes the newest answer to a paired lens. Supabase is used strictly as the identity provider. Pairing and display state remain process-local.

LOCAL API

Running and healthy

GITHUB CI/CD

Passing

HOSTED BACKEND

Not deployed

AUTOMATED TESTS

21 passing

PRODUCTION IMAGE

Published to GHCR

HOSTED SUPABASE PROJECTS

Production connected; trial pending

- What is ready

The API contract, local runtime, JWT verification, OpenAI request path, error mapping, in-memory pairing flow, container image, GitHub validation, container publishing, and protected main branch are operational.
- Primary production blocker

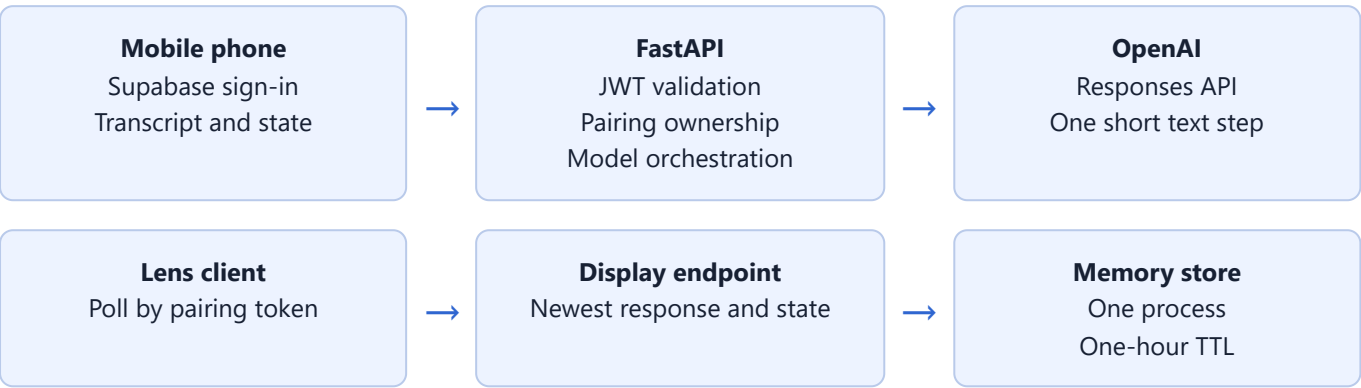
There is no persistent backend hosting target, the trial environment is not configured, and pairing state is lost on process restart. A single replica is mandatory until state moves to a shared TTL store.

Contents

1	System purpose and architecture
2	API contract and request lifecycle
3	Supabase authentication
4	OpenAI model integration
5	Pairing state and data lifecycle
6	Configuration and environments
7	Container, GitHub CI/CD, and repository controls
8	Testing and quality status
9	Security assessment
10	Remaining work and production roadmap

1. System purpose and architecture

The service supports a deliberately narrow interaction model. A phone owns conversation history and voice interaction, while the lens receives only short text and a simple activity state. The backend is the trust boundary between authenticated phone traffic, the OpenAI model provider, and unauthenticated lens polling.



Component map

Component	Responsibility	Implementation
<code>app/main.py</code>	Application factory, middleware, routes, status-code mapping, dependency composition	FastAPI
<code>app/models.py</code>	Request and response validation, API naming aliases, state vocabulary	Pydantic
<code>app/auth.py</code>	Bearer extraction and Supabase access-token verification	PyJWT and remote JWKS
<code>app/service.py</code>	Prompt policy and OpenAI Responses API call	Async OpenAI SDK
<code>app/store.py</code>	Pairing ownership, state changes, expiration, newest display response	Async in-memory dictionary
<code>app/config.py</code>	Environment parsing and unsafe-configuration rejection	Typed dataclass

Explicit scope boundaries

- The backend does not accept, store, or return audio.
- The backend does not retain the conversation transcript.
- The backend does not expose login, signup, or refresh-token endpoints.
- The backend does not use Supabase Postgres, Storage, Realtime, or service-role credentials.
- The backend does not currently integrate NVIDIA APIs or the previously supplied NVIDIA example.
- The backend returns one complete text response rather than streaming partial output.

2. API contract and request lifecycle

Endpoint summary

Method and path	Authentication	Purpose	Main responses
GET /healthz	None	Reports process status, selected environment, and auth mode	200
POST /v1/state	Supabase bearer token outside local mode	Registers a pairing or changes lens state	204, 401, 403, 422
POST /v1/chat	Supabase bearer token outside local mode	Generates and caches the next display instruction	200, 401, 403, 413, 422, 429, 503
GET /v1/display?token=...	Pairing-token capability	Returns newest text and current state to the lens	200, 401, 422

Interactive Swagger documentation

Swagger UI is available at `/docs`, ReDoc is available at `/redoc`, and the machine-readable OpenAPI document is available at `/openapi.json`. All four endpoints appear in the schema.

- **System** contains process health and runtime-mode visibility.
- **Phone** contains authenticated chat and state operations.
- **Lens** contains capability-protected display polling.
- Every operation has a stable operation ID, summary, detailed description, and response description.
- Request and response fields include human-readable descriptions and representative examples.
- Application-level 401, 403, 413, 429, and 503 responses are documented with a shared error schema.
- The authorization dialog identifies the credential as a Supabase bearer access token.
- Filtering, alphabetical tag sorting, method sorting, request duration, and authorization persistence are enabled.

Input validation

- A pairing token must contain exactly 32 hexadecimal characters, providing 128 bits when generated randomly.
- A chat request must contain at least one message.
- Message roles are limited to `user` and `assistant`.
- Message text cannot be empty or whitespace-only.
- The serialized transcript cannot exceed 256,000 UTF-8 bytes.
- State must be `idle`, `listening`, `thinking`, or `speaking`.
- External JSON uses camelCase while Python uses snake_case.

Chat sequence

1. FastAPI and Pydantic validate the bearer header, pairing token, messages, and transcript size.
2. The store registers or verifies pairing ownership and changes the state to `thinking`.
3. The chat service submits the entire supplied transcript and fixed system instruction to OpenAI.
4. A provider rate limit becomes HTTP 429. A missing key, API error, or empty model output becomes HTTP 503.
5. On provider failure, the state returns to `idle`.
6. On success, a new response ID and UTC timestamp are created, the state becomes `speaking`, and the newest text is saved.
7. The API returns that response to the phone. The lens receives the same value on its next poll.

Representative payloads

```
POST /v1/chat
Authorization: Bearer <supabase-access-token>
Content-Type: application/json

{
  "pairingToken": "a3f9d1e2c3b4a5f60718293a4b5c6d7e",
  "messages": [
    {"role": "user", "content": "How do I change a bike tire?"}
  ]
}
```

```
{
  "responseId": "r_<random-uuid>",
  "text": "Use your tire levers to lift one side of the tire bead over the rim.",
  "createdAt": "2026-09-04T18:00:00Z"
}
```

Capability endpoint

`/v1/display` is intentionally not protected by Supabase. Knowledge of the random pairing token grants read access to its latest display state. Because the token is a URL query parameter, infrastructure logs and browser history must be treated carefully.

3. Supabase authentication

Delivered behavior

The mobile client is expected to authenticate directly with Supabase. It sends the current access token to protected FastAPI routes. The backend uses the configured Supabase URL to retrieve public signing keys and verifies tokens locally. The official Supabase JWT guide documents the project JWKS endpoint and recommends established JWT libraries for verification.

JWT control	Current enforcement
Signature	ES256 or RS256 only
Issuer	{SUPABASE_URL}/auth/v1
Audience	authenticated by default, configurable
Required claims	exp , iat , and sub
Role	Must equal authenticated
User identity	JWT subject, with optional email claim
Key caching	Public JWKS cached for 600 seconds
Unauthorized response	401 with WWW-Authenticate: Bearer

Security choices already made

- No user password enters FastAPI.
- No secret or service-role Supabase key is needed by FastAPI.
- Legacy shared-secret HS256 tokens are deliberately rejected.
- Authentication bypass is allowed only with APP_ENV=local .
- Trial and production require HTTPS Supabase URLs.
- The first authenticated user to register a pairing token owns it until expiration.
- User-editable metadata is not used for authorization decisions.

Local Supabase stack

Service	Location	Purpose
API and Auth	http://127.0.0.1:54321	Development authentication
Postgres	Port 54322	Available locally but unused by this application
Studio	Port 54323	Local administration UI
Email viewer	Port 54324	Captures local authentication email

Email signup and refresh-token rotation are enabled locally. Email confirmation is disabled locally, anonymous sign-in is disabled, JWT lifetime is one hour, and the native callback is glance-local://auth/callback . Each developer generates an ignored ES256 signing-key file.

Not yet completed

- The trial project reference remains a placeholder.

- Production email Auth is reachable, but Google and Apple providers are not enabled.
- Hosted redirect allowlists, site URLs, and email policies still require end-to-end verification.
- The mobile sign-in, Keychain storage, refresh flow, and deep-link handling are outside this repository and remain integration work.
- There is no live Supabase token integration test in CI.
- Immediate session revocation is not checked against Supabase on each API request.

4. OpenAI model integration

Delivered request configuration

Setting	Current value	Intent
API	Responses API	Modern OpenAI response interface
Client	<code>openai.AsyncOpenAI</code>	Non-blocking route execution
Default model	<code>gpt-5.6-sol</code>	High-quality instruction generation
Reasoning	<code>low</code>	Favor response latency for one-step guidance
Output budget	160 tokens	Constrain lens output
Response storage	<code>false</code>	Do not retain responses for later API retrieval
Prompt cache key	<code>metaglasses-v1</code>	Improve reuse for common prompt prefixes
Safety identifier	SHA-256 hash of pairing token	Stable pseudonymous identifier without sending raw token

Official OpenAI documentation confirms that GPT-5.6 Sol supports the Responses API, text input and output, image input, streaming, structured output, and reasoning efforts including low. The application currently uses only text input and complete text output.

Prompt policy

The fixed instruction defines the assistant as a step-by-step voice assistant and requires exactly one practical next step in one short plain-text sentence. Markdown, preambles, and follow-up questions are discouraged except when safety requires clarification.

Data behavior

- The phone supplies the complete conversation history on each chat request.
- The backend forwards that supplied transcript to OpenAI.
- The backend does not write the transcript to its own store.
- The raw pairing token is hashed before use as the safety identifier.
- The generated response is stored only in process memory for lens retrieval.

Provider failure mapping

Condition	Backend result
<code>OPENAI_API_KEY</code> missing	503 Model provider unavailable
OpenAI rate limit	429 with retry guidance
Other OpenAI API error	503
Empty model output	503

Current local limitation

The local environment has no OpenAI key configured. Health and pairing endpoints work, but a real local chat request returns HTTP 503 until a valid key is supplied outside version control.

5. Pairing state and data lifecycle

The store is an asynchronous, process-local dictionary protected by one lock. It is sufficient for deterministic v0.1 behavior and unit testing, but it is not a durable or horizontally scalable storage layer.

Pairing record

Field	Meaning
<code>owner_id</code>	Authenticated Supabase user subject that claimed the pairing
<code>state</code>	Current lens state
<code>last_phone_activity</code>	UTC time used for expiration
<code>response_id</code>	Random UUID-based identifier prefixed with <code>r_</code>
<code>text</code>	Newest generated answer only
<code>created_at</code>	UTC creation time of newest answer

Expiration and ownership

- Default time to live is 3,600 seconds.
- Expired records are removed lazily when another store operation occurs.
- A new phone request refreshes the activity time.
- A different user attempting to use an active pairing token receives HTTP 403.
- An unknown or expired display token receives HTTP 401 and should trigger re-pairing.

Operational consequences

Single-process constraint

Restarting or redeploying the service deletes every pairing. Multiple replicas would hold different records, so the phone and lens could observe inconsistent state. Production must run one replica until a shared TTL store is introduced.

The present design also has no durable audit trail, historical conversation store, user profile store, or analytics store. This is intentional for v0.1, but must be reflected in product expectations and deployment settings.

6. Configuration and environments

Environment variables

Variable	Required when	Purpose
APP_ENV	Always	Selects local, trial, or prod
AUTH_MODE	Defaults to required	Allows required or disabled authentication
SUPABASE_URL	Authentication required	Defines issuer and public JWKS host
SUPABASE_JWT_AUDIENCE	Optional	Defaults to authenticated
CORS_ORIGINS	Always	Comma-separated client origin allowlist
OPENAI_API_KEY	Real chat requests	Server-side OpenAI credential
OPENAI_MODEL	Optional	Defaults to GPT-5.6 Sol
PAIRING_TTL_SECONDS	Optional	Defaults to 3,600 seconds

Environment matrix

Environment	Authentication	Supabase	Backend location	Status
Local	Disabled or required	None or local CLI at 127.0.0.1:54321	http://127.0.0.1:8000	Templates available
Trial	Required	Hosted project placeholder	Not deployed	Not configured
Prod	Required	Hosted project connected	Not deployed	Email Auth reachable

Configuration guardrails

- APP_ENV must be selected explicitly.
- Authentication cannot be disabled in trial or prod.
- A Supabase URL is mandatory when authentication is required.
- Supabase URLs must be absolute HTTP or HTTPS URLs.
- Trial and production require HTTPS.
- CORS must contain at least one origin.
- Trial and production reject wildcard CORS.

CORS policy

The application allows GET and POST requests with Authorization and Content-Type headers. Cookie credentials are disabled. Local mode accepts all origins, while hosted environments require explicit origins.

7. Container, GitHub CI/CD, and repository controls

Production container

- Uses Python 3.12 slim.
- Copies uv 0.12.9 from the official uv image.
- Installs the frozen lockfile without development dependencies.
- Runs under a non-root system user.
- Exposes port 8000.
- Checks `/healthz` every 30 seconds.
- Starts Uvicorn on `0.0.0.0:8000`.
- Excludes Git data, environments, caches, tests, docs, and Supabase development files from the build context.

GitHub Actions validation

The workflow runs for pull requests into main, pushes to main, and manual dispatch. Older work on the same ref is cancelled through concurrency control.

1. Check out the repository using a commit-pinned action.
2. Install Python 3.12.
3. Install uv 0.12.9 with dependency caching.
4. Install all locked development dependencies.
5. Run Ruff.
6. Run all pytest tests.
7. Build the production Docker image.

Container publishing

After validation succeeds on main, a dependent job signs in to GitHub Container Registry with the temporary workflow token and publishes:

```
ghcr.io/adipat2003/metaglasses-backend:latest
ghcr.io/adipat2003/metaglasses-backend:<full-git-commit-sha>
```

BuildKit layer caching is stored in GitHub Actions. Pulling a private package from a hosting provider will require registry credentials unless package visibility is changed.

Repository governance

Control	Status
Repository visibility	Public
Default branch	Main
Pull requests required	Yes
Required status check	Validate
Strict up-to-date check	Yes
Administrator enforcement	Yes
Linear history	Required

Control	Status
Conversation resolution	Required
Force pushes	Disabled
Branch deletion	Disabled

Deployment distinction

GitHub builds and stores the deployable image, but it does not currently run the FastAPI service. GitHub Pages cannot host a Python process. A container runtime such as a managed container platform, virtual machine, or self-managed server is still required.

8. Testing and quality status

Verified results

RUFF Passed	PYTEST 20 passed
GITHUB DOCKER BUILD Passed	LIVE LOCAL HEALTH HTTP 200

Covered behavior

- Unknown display tokens and re-pair response.
- Phone state registration and display propagation.
- Missing, invalid, and accepted bearer credentials.
- OpenAPI security declarations.
- Cross-user pairing ownership rejection.
- Local health and auth-bypass reporting.
- Chat generation and lens response consistency.
- Transcript byte-size enforcement.
- Rate-limit and provider-failure status mapping.
- OpenAI model, reasoning, and response-storage options.
- Environment, CORS, Supabase URL, and HTTPS validation.

Testing gaps

- No real Supabase login and JWT smoke test.
- No real OpenAI request in CI.
- No end-to-end phone-to-lens test.
- No concurrency or load testing.
- No coverage threshold or coverage report.
- No static type checker.
- No dependency or container vulnerability scan.
- No deployment health gate or rollback exercise.

Two non-failing dependency deprecation warnings currently originate from Starlette TestClient and its HTTPX/AnyIO integration. They should be tracked before future dependency upgrades.

9. Security assessment

Strengths

Area	Control
Secrets	Real environment files and generated signing keys are ignored; no service-role key is needed.
Identity	Asymmetric JWT verification checks issuer, audience, lifetime, subject, and role.
Authorization	Pairing ownership prevents a second authenticated user from reusing an active token.
Hosted configuration	Authentication bypass and wildcard CORS are rejected outside safe environments.
Provider privacy	OpenAI response storage is disabled and pairing identifiers are hashed.
Container	Runs as non-root with production dependencies only.
Supply chain	Python lockfile is committed and GitHub Actions are pinned to full commits.
Source control	Main is protected from direct unvalidated history changes.

Risks and recommended controls

Risk	Impact	Recommended response
Pairing token in query string	Token may enter logs or history	Redact query parameters in infrastructure logs; consider a path/header protocol in a later client version.
No application rate limiting	Cost abuse and provider exhaustion	Add per-user and per-IP limits before public launch.
In-memory state	Loss on deploy and incorrect multi-replica behavior	Use a shared TTL store or enforce one replica with explicit operational monitoring.
Health endpoint is shallow	Process can be healthy while dependencies fail	Keep liveness shallow; add a separate readiness endpoint for required dependencies.
No centralized observability	Slow detection and diagnosis	Add structured logs, request IDs, metrics, alerting, and error reporting.
No automated vulnerability scan	Known dependency or image flaws may go unnoticed	Add dependency review, secret scanning verification, and container scanning.
Access tokens remain valid until expiry	Deleted or revoked users may retain short-lived access	Keep token lifetimes appropriate; add session checks only for operations requiring immediate revocation.

10. Remaining work and production roadmap

Priorities below separate launch blockers from scalability and product enhancements. P0 is required before a credible public production launch. P1 is strongly recommended for an external trial. P2 and P3 improve resilience, maintainability, and product capability.

Priority	Work item	Required outcome	Verification
P0	Select a runtime host	One persistent container instance with HTTPS and stable public URL	Public health request returns 200
P0	Create trial Supabase project	Real URL, ES256 signing key, email policy, site URL, and redirect allowlist	Real trial JWT accepted
P0	Configure hosted secrets	APP_ENV, AUTH_MODE, Supabase URL, CORS, OpenAI key, model, and TTL stored by host	No secret in image or repository
P0	Add deployment job	Successful main build deploys immutable image to trial	Deployment status and health gate pass
P0	Connect mobile auth	Direct Supabase sign-in, secure refresh-token storage, refresh, and bearer injection	Device end-to-end test passes
P0	Protect pairing capability	Random generation, secure transfer, log redaction, and expiry behavior documented	Logs contain no full token
P1	Add rate limiting	Per-user and per-IP budgets around chat requests	Deterministic 429 tests
P1	Add observability	Structured logs, request IDs, latency, error rate, provider failures, alerts	Dashboard and alert test
P1	Add live integration checks	Controlled Supabase JWT and OpenAI smoke tests outside pull requests	Scheduled workflow passes
P1	Define rollback	Immutable image deployment and documented rollback command	Rollback drill passes
P1	Harden CI	Coverage threshold, type checking, dependency review, and image scan	Required checks enforced
P2	Shared pairing store	Durable or ephemeral shared TTL state with atomic ownership operations	Multi-replica test passes
P2	Readiness and shutdown	Dependency readiness, graceful shutdown, and deployment drain behavior	Failure-mode test passes
P2	Load and cost testing	Latency, throughput, memory, and OpenAI cost envelope documented	Target service levels met
P3	Streaming	Optional partial-response delivery if product latency requires it	Client compatibility test

Priority	Work item	Required outcome	Verification
P3	Multimodal input	Explicit image or audio contract if product requirements expand	Privacy and cost review

Suggested delivery phases

Phase 1: Trial deployment

- Select a container host and provision exactly one replica.
- Create and configure the hosted trial Supabase project.
- Store environment values in the hosting platform.
- Extend GitHub Actions to deploy the commit-tagged image.
- Implement mobile authentication and run a complete device-to-lens test.

Phase 2: Operational hardening

- Add rate limiting, logging, request correlation, metrics, and alerts.
- Add readiness checks, rollback procedures, integration tests, and vulnerability scans.
- Define service-level objectives for availability, response latency, and provider failure rates.

Phase 3: Production and scale

- Create an isolated production Supabase project and production hosting service.
- Move pairing state into a shared TTL system before adding replicas.
- Perform load, security, privacy, recovery, and cost reviews.
- Promote immutable images through trial to production with an approval gate.

Production-readiness checklist

Done	Requirement	Current evidence or next action
☒	API contract implemented	Four documented FastAPI endpoints
☒	JWT authentication implemented	Supabase public JWKS verification
☒	Provider integration implemented	OpenAI Responses API with tested options
☒	Automated unit tests	20 passing
☒	Container build	GitHub runner build passed
☒	Protected source branch	Required Validate check and pull requests
☐	Live trial host	Select provider and provision service
☐	Real hosted Supabase projects	Create trial and production projects
☐	Hosted secret configuration	Set values in runtime secret manager
☐	Automated runtime deployment	Add provider-specific GitHub job
☐	Mobile authentication integration	Implement in mobile repository
☐	Rate limiting and abuse protection	Choose and implement policy
☐	Production observability	Add logs, metrics, errors, and alerts
☐	Shared pairing state	Required before horizontal scaling
☐	Recovery and rollback validation	Document and test rollback
☐	Security and load review	Complete before production launch

Useful local commands

```
# Start without authentication
uv run uvicorn app.main:app --reload --env-file .env.local

# Validate
uv run ruff check .
uv run python -m pytest

# Local endpoints
http://127.0.0.1:8000/healthz
http://127.0.0.1:8000/docs
```

References

- Repository: github.com/Adipat2003/metaglasses-backend
- Successful CI/CD run: [GitHub Actions run 3390493502](https://github.com/Adipat2003/metaglasses-backend/actions/runs/3390493502)
- Supabase JWT guide: supabase.com/docs/guides/auth/jwts
- OpenAI Docs, GPT-5.6 Sol: developers.openai.com/api/docs/models/gpt-5.6-sol
- GitHub Pages scope: [docs.github.com Pages overview](https://docs.github.com/pages/overview)
- GitHub Container Registry: [docs.github.com Container registry](https://docs.github.com/containers/registry)

This report describes the verified state on September 4, 2026. Hosted URLs, external secrets, provider account configuration, mobile-client implementation, and later repository changes are outside this snapshot unless explicitly stated.